Experiment No: 3

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created

Tools & Libraries used :

- Install ***Flask*** : pip install Flask
- Download ***Postman*** from https://www.postman.com/
- Python Libraries : **datetime, jsonify, hashlib, uuid4, urlparse, request**
- **Install requests :** pip install requests==2.18.4

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.
Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).
Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
  pip install Flask
- **Requests** – For HTTP communication between nodes
  pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
  - datetime
  - jsonify
  - hashlib
  - uuid4
  - urlparse
  - request

Procedure and Output:

1. Download hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py.
2. Install required packages.
3. Run each node in separate terminals.

```
PS C:\Users\mahek\Downloads\blockchain exp3> cd "C:\Users\mahek\Downloads\blockchain exp3"
PS C:\Users\mahek\Downloads\blockchain exp3> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\mahek\Downloads\blockchain exp3> python hadcoin_node_5001.py
 * Serving Flask app 'hadcoin_node_5001'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://192.168.1.106:5001
Press CTRL+C to quit
```

```
PS C:\Users\mahek\Downloads\blockchain exp3> cd "C:\Users\mahek\Downloads\blockchain exp3"
PS C:\Users\mahek\Downloads\blockchain exp3> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "C:\w

(venv) PS C:\Users\mahek\Downloads\blockchain exp3> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\mahek\Downloads\blockchain exp3> python hadcoin_node_5002.py
 * Serving Flask app 'hadcoin_node_5002'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://192.168.1.106:5002
Press CTRL+C to quit
```

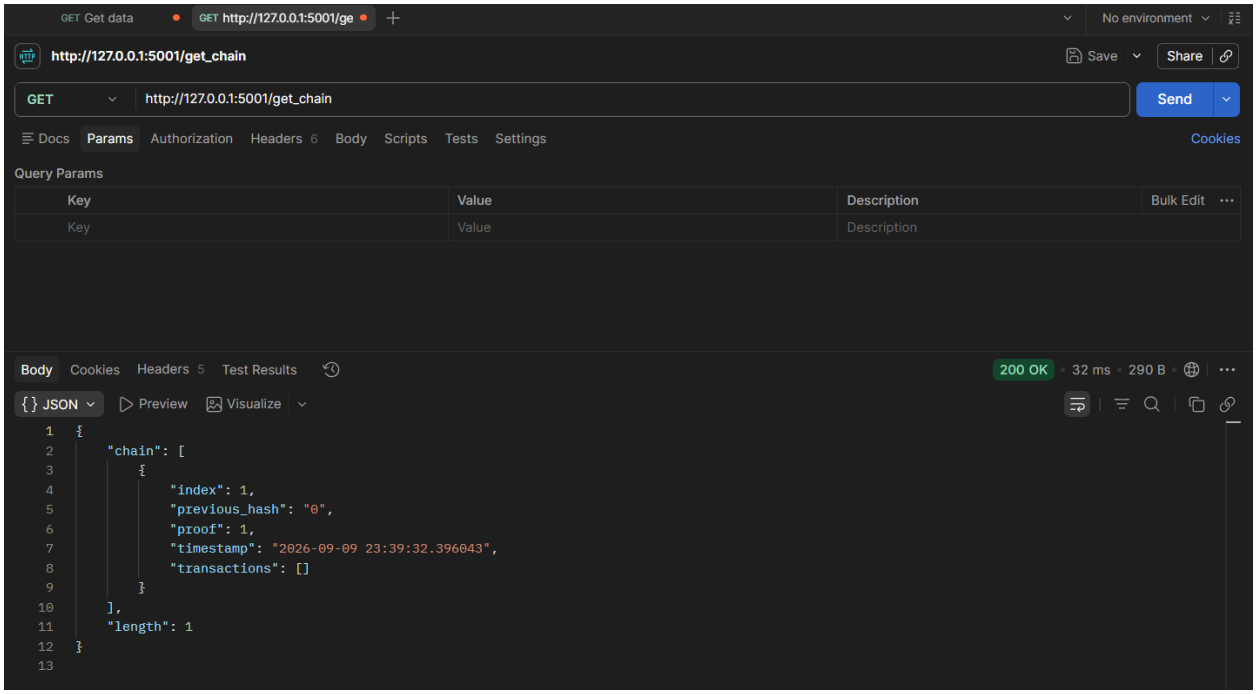
```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\mahek\Downloads\blockchain exp3> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "C:\Users\mahe

(venv) PS C:\Users\mahek\Downloads\blockchain exp3> python hadcoin_node_5003.py
 * Serving Flask app 'hadcoin_node_5003'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5003
 * Running on http://192.168.1.106:5003
Press CTRL+C to quit
```

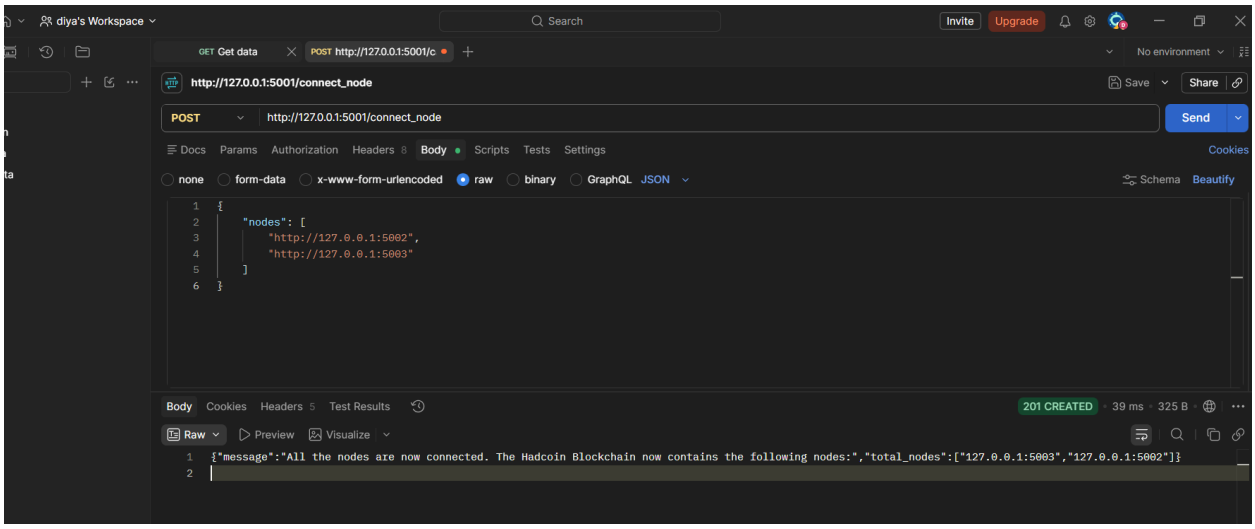
4. Connect nodes using Postman – POST /connect_node.

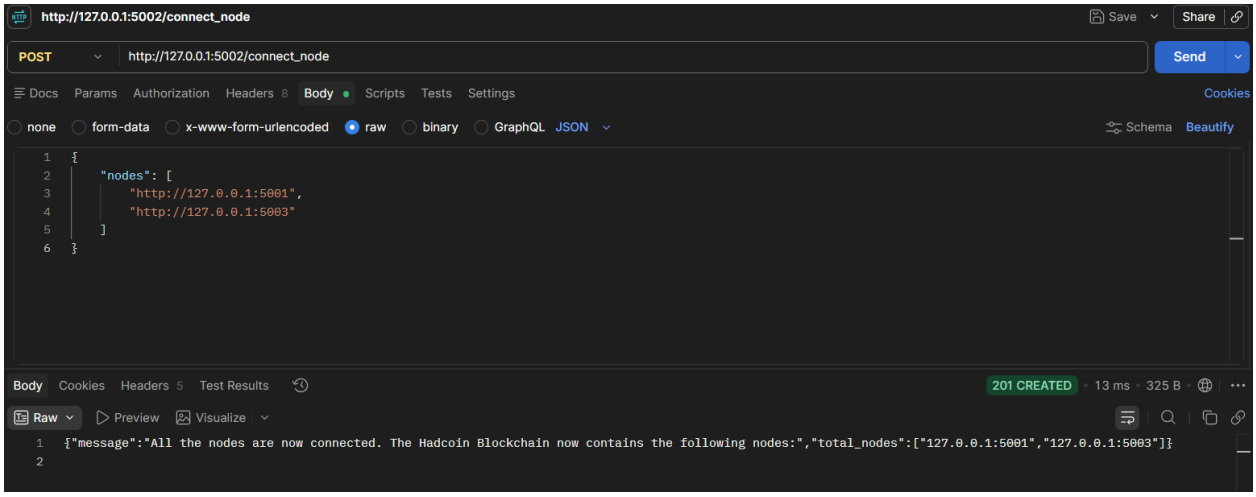
{ "nodes": ["http://127.0.0.1:5002", "http://127.0.0.1:5003"] }

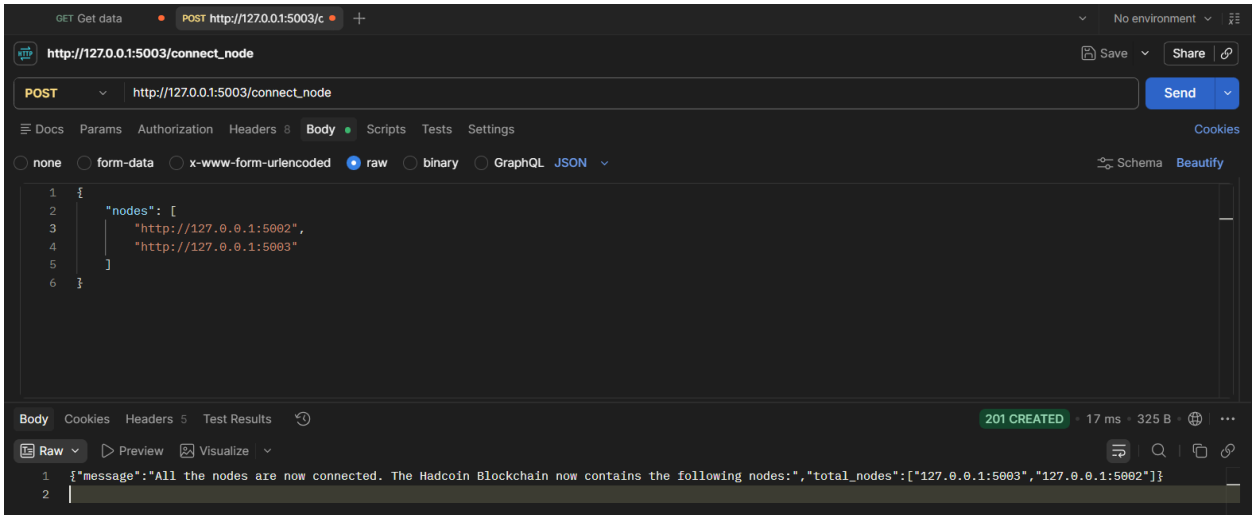
5.



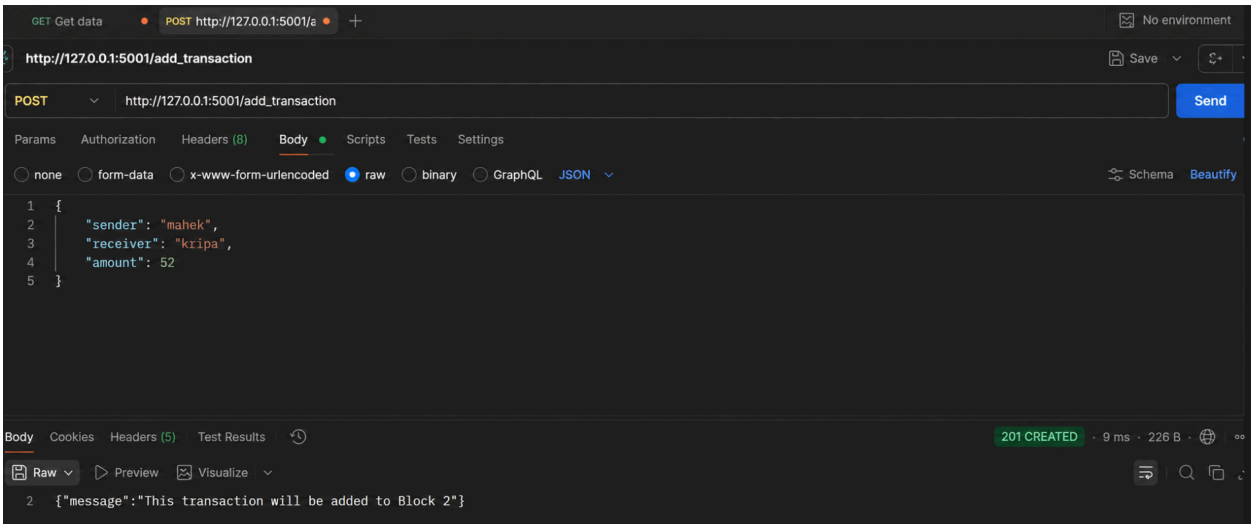
6. Add transactions – POST -

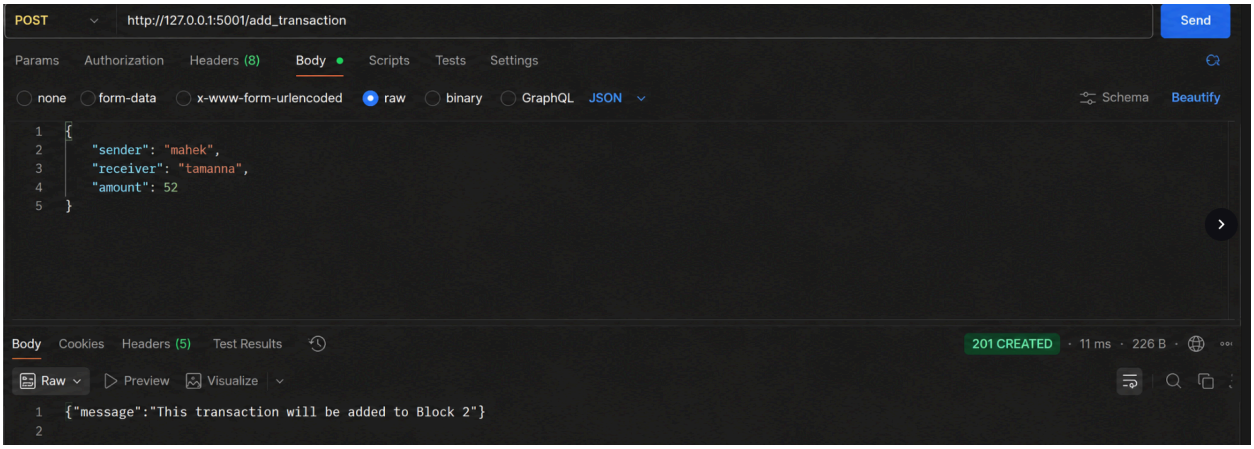


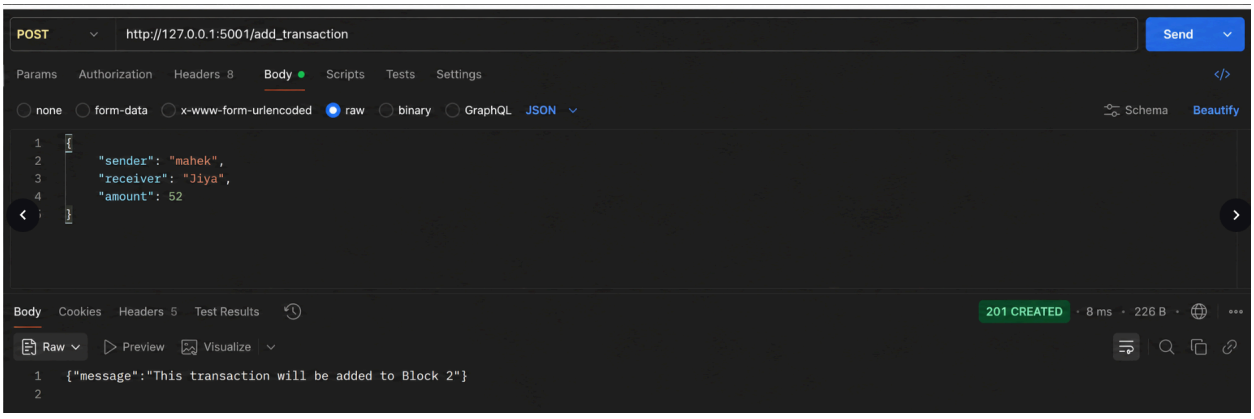


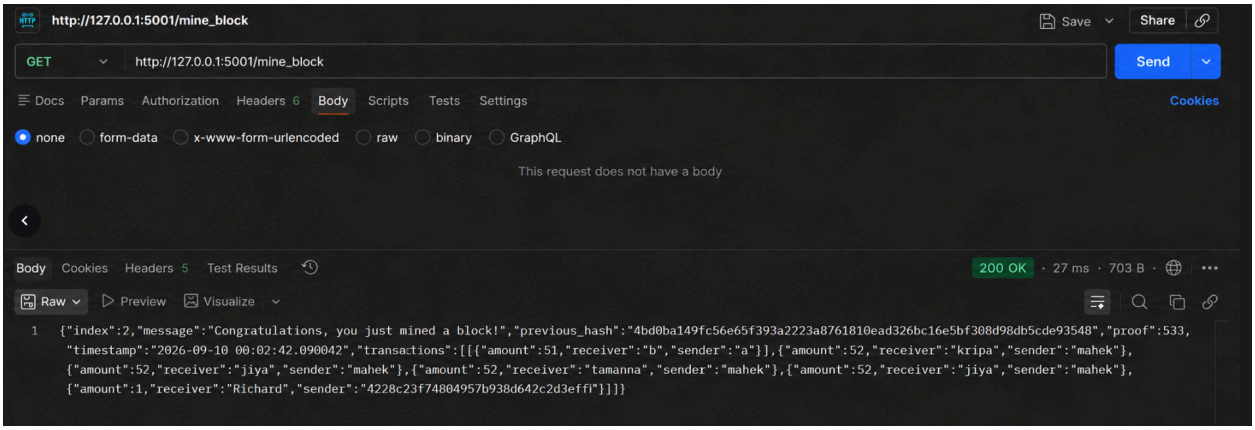


6. Add 3 transactions in 5001


GET Get data
POST http://127.0.0.1:5001/a
No environment
http://127.0.0.1:5001/add_transaction
Save
POST
http://127.0.0.1:5001/add_transaction
Send
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"sender": "mahek",
"receiver": "kripa",
"amount": 52
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
9 ms
226 B
Raw
Preview
Visualize
{"message":"This transaction will be added to Block 2"}


POST
http://127.0.0.1:5001/add_transaction
Send
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"sender": "mahek",
"receiver": "tamanna",
"amount": 52
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
11 ms
226 B
Raw
Preview
Visualize
{"message":"This transaction will be added to Block 2"}


POST
http://127.0.0.1:5001/add_transaction
Send
Params
Authorization
Headers 8
Body
Scripts
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"sender": "mahek",
"receiver": "Jiya",
"amount": 52
}
Body
Cookies
Headers 5
Test Results
201 CREATED
8 ms
226 B
Raw
Preview
Visualize
{"message":"This transaction will be added to Block 2"}



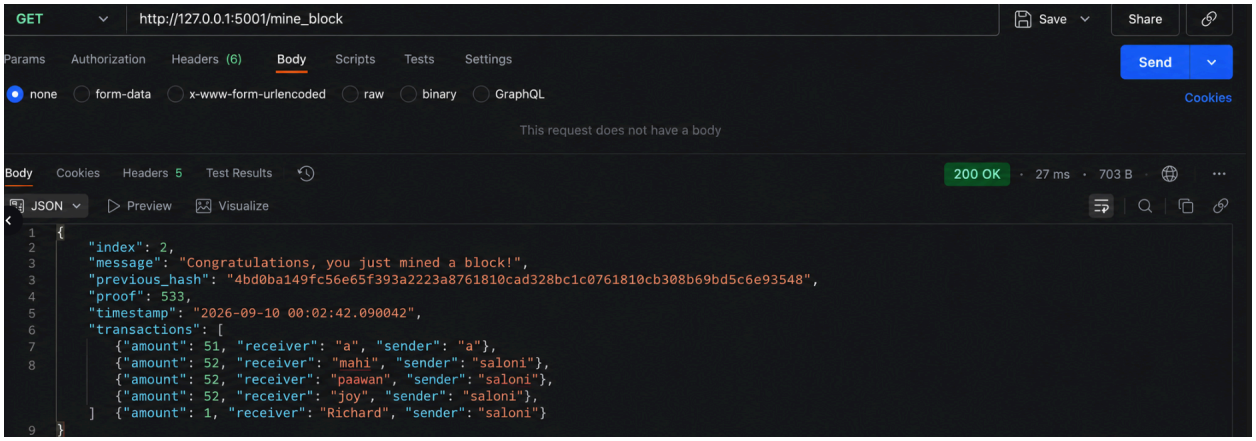
7. Condition 2 — Each node must have **different chain lengths**

STEP 2 — Perform all transactions ONLY from one node (e.g., 5001)

POST → http://127.0.0.1:5001/add_transaction

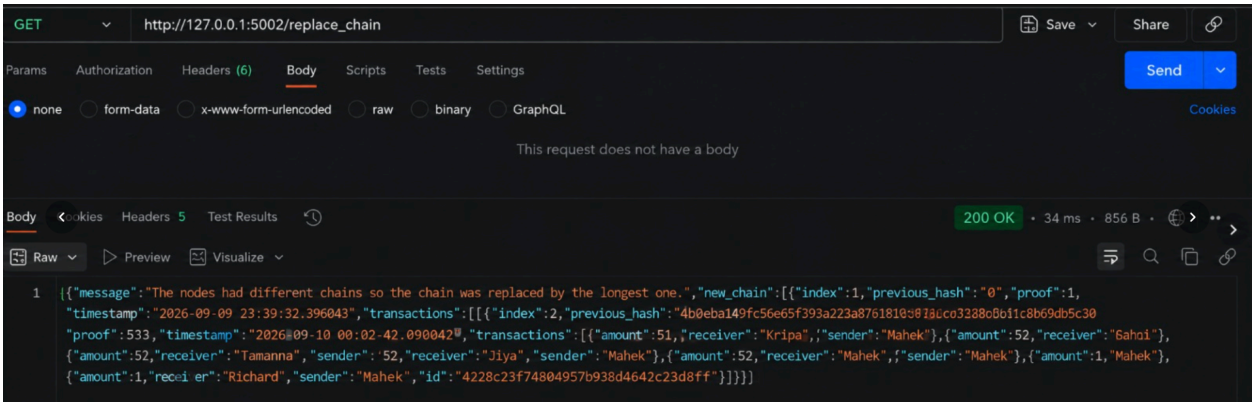
{"sender": "saloni", "receiver": "mahi", "amount": 52 }

Then 5001 → GET /mine_block



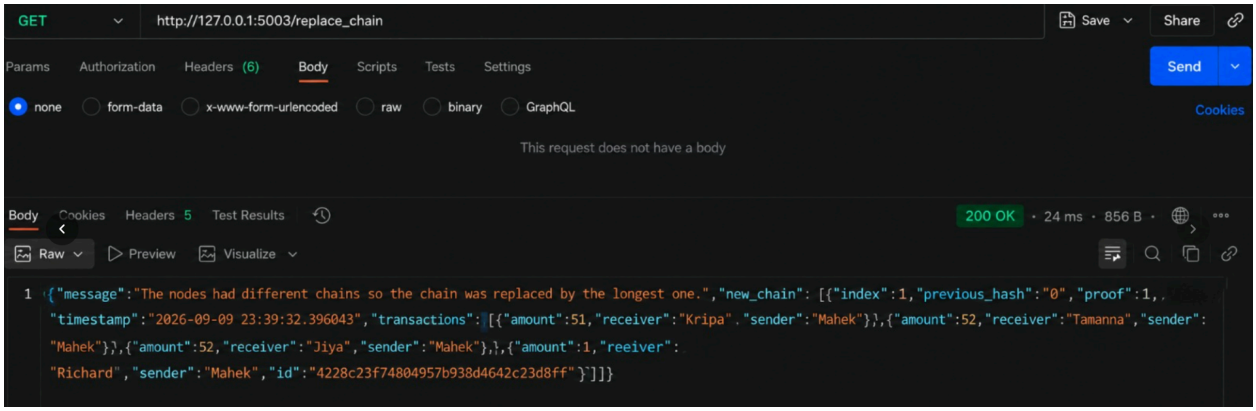
STEP 3 — Now go to 5002 and run:

GET → /replace_chain



STEP 4 — Repeat on 5003

GET → /replace_chain



Code Modification — Remove Transactions After Mining

Original code:

self.chain.append(block)

return block

Modified code:

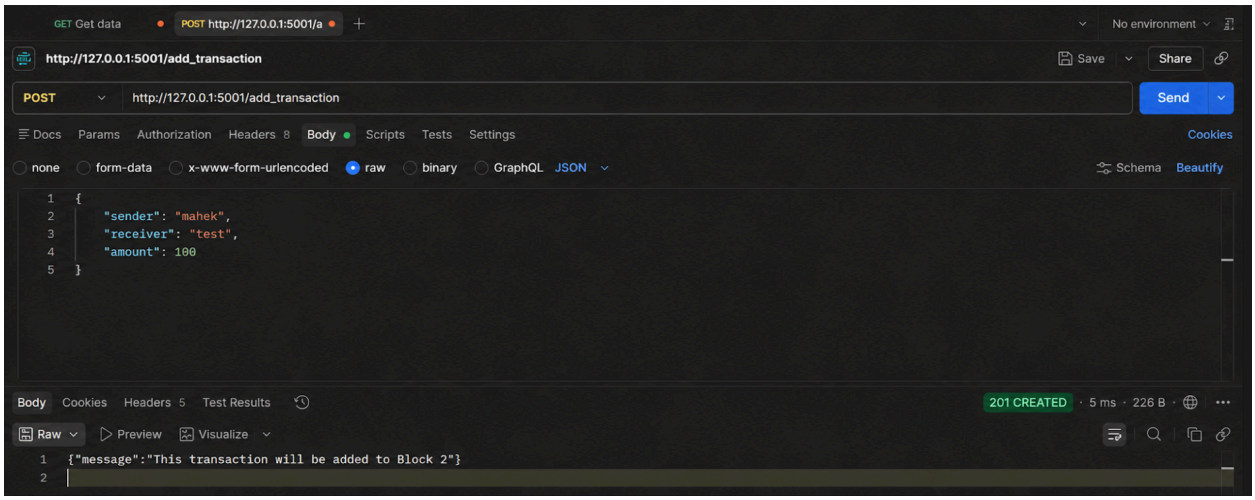
self.chain.append(block)

self.transactions = []

return block

```python
hadcoin_node_5001.py
hadcoin_node_5001.py > Blockchain
class Blockchain:
    def __init__(self):
        self.nodes = set()

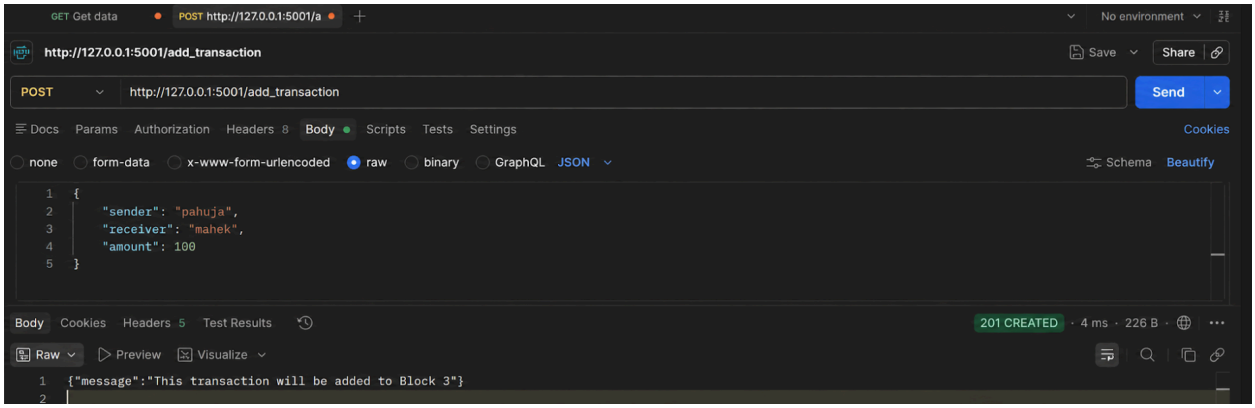
    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': self.transactions}
        self.transactions = []
        self.chain.append(block)
        self.transactions = []
        return block
```



{ "sender": "mahek", "receiver": "test", "amount": 100 }

Output

{"index":2,"message":"Congratulations, you just mined a block!","previous_hash":"a6a3ae9b6352a22c808d8f1fc7d7c2586177830ea1c1ce2de48f5262404624ca","proof":533,"timestamp":"2026-09-10 00:20:14.324398","transactions":[{"amount":100,"receiver":"test","sender":"mahek"},{"amount":1,"receiver":"Richard","sender":"4ab9f79c91d74c88aca347ba55c72692"}]}

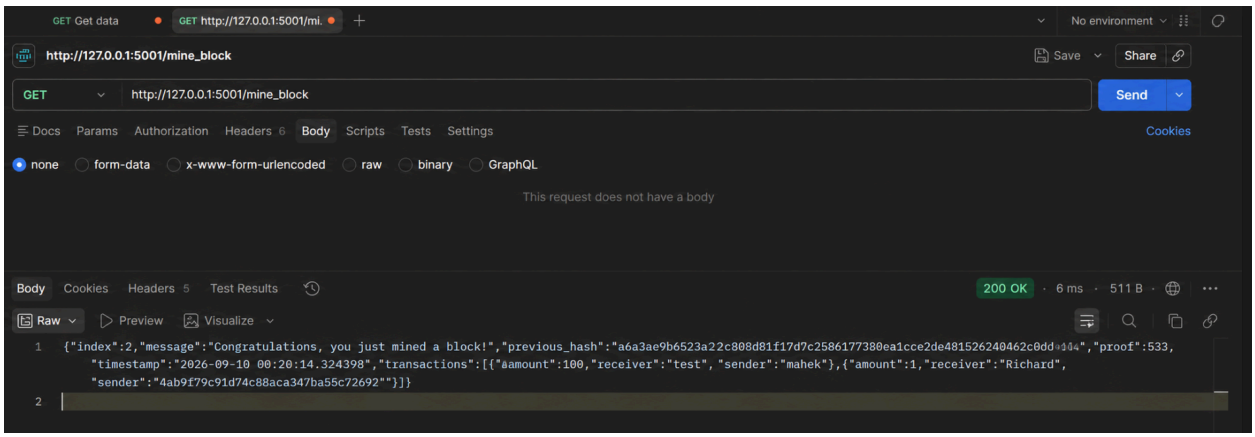


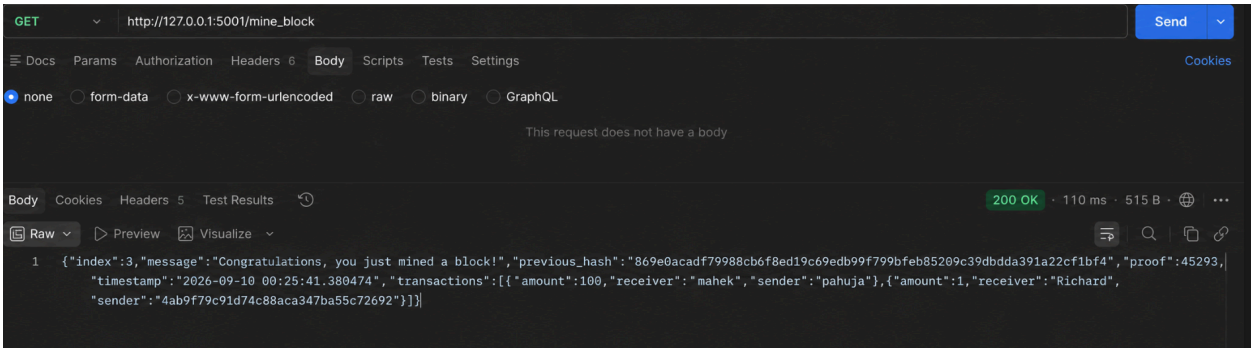
{ "sender": “pahuja", "receiver": "mahek", "amount": 100 }

Output - {"index":3,"message":"Congratulations, you just mined a block!","previous_hash":"869e0acadf79088cb6f8ed19c69edb99f799bfe852d9c39dbdda391a2c2f1bf4","proof":45293,"timestamp":"2026-09-10 00:25:41.380474","transactions":[{"amount":100,"receiver":"mahek","sender":"pahuja"},{"amount":1,"receiver":"Richard","sender":"4ab9f79c91d74c88aca347ba55c72692"}]}

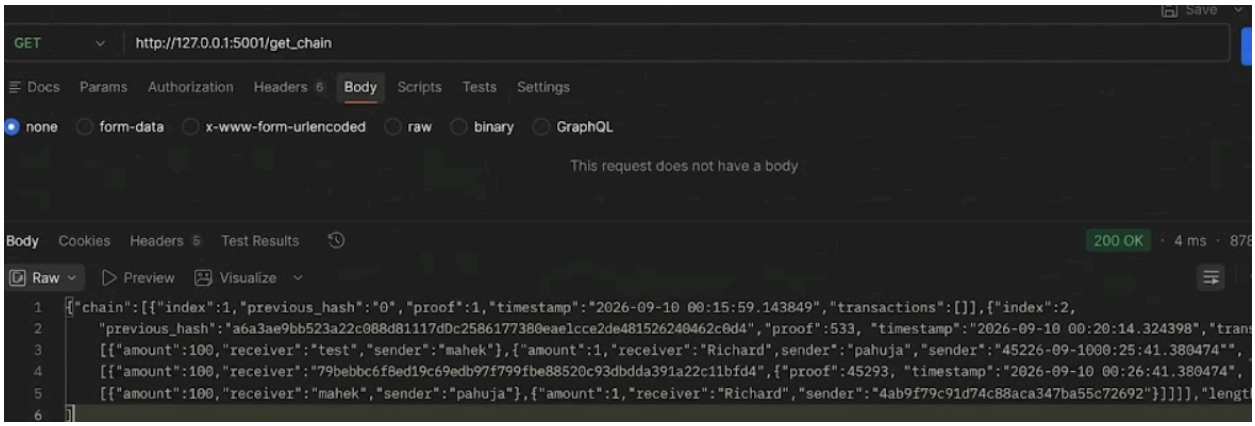
After a transaction is added to a block, the pending transaction list is cleared. Therefore, the transaction from Block 2 is not included again when Block 3 is mined.

8. Mine blocks – GET /mine_block.





9. Fetch blockchain – GET /get_chain.



Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.